Revision — 1.3.4 Web Technologies

OCR H446 — one-page revision summary

Must know

- **HTML** defines the **structure and content** of a page (headings, paragraphs, links, images).
- **CSS** defines **presentation/style** (colour, font, layout). Selectors target a tag (bare name), a `.class`, or an `#id`.
- **JavaScript** provides **interactivity/behaviour**, usually **client-side** (events, form validation, dynamic updates without reloading).
- **Search indexing**: a web crawler/spider follows links and reads pages; the engine builds an **index** mapping keywords → the pages they appear on for fast retrieval, then ranks results by relevance.
- **PageRank** ranks pages by *importance* based on incoming links: each link is a "vote", links from high-rank pages are worth more, and a page divides its rank across its outgoing links.
- The **damping factor** (~0.85) models a "random surfer" who keeps following links rather than jumping randomly; it stops ranks looping to unrealistic values.
- PageRank is calculated **iteratively** — ranks are recalculated repeatedly using the latest values — and stops when the values **converge** (stop changing significantly).
- **Client-side** processing runs in the user's **browser**: fast/immediate, reduces server load, but **insecure** (can be viewed/bypassed) and depends on the device.
- **Server-side** processing runs on the **web server**: secure and hidden, but slower (round trip) and adds server load. Security-critical checks (login, payment, database queries) **must** run server-side; client-side validation alone can be bypassed.

Key terms

Term	Definition
HTML	Language defining a web page's structure and content
CSS	Language defining presentation/style (colour, font, layout)
JavaScript	Language adding client-side interactivity and behaviour
Web crawler/spider	Program that follows links and reads pages to build the index
Search index	Store mapping keywords to the pages they appear on
PageRank	Algorithm ranking pages by importance from incoming links
Damping factor	~0.85 value modelling a random surfer; ensures ranks converge
Client-side / server-side	Processing on the user's browser / on the web server

How to — CSS selectors and choosing where to process

Selectors: `p { }` targets all `<p>` tags; `.warning { }` targets elements with `class="warning"`; `#header { }` targets the single element with `id="header"`.

Client vs server example (online checkout): - Checking an **email format** is simple and non-critical → do it **client-side** for instant feedback and no server round trip (reduces load). - Verifying **payment/login** is security-critical → do it **server-side**, because client-side code can be viewed and bypassed, and the bank's logic/records must stay hidden on the server.

Exam tips

- Keep the three roles distinct: HTML = structure, CSS = style, JavaScript = behaviour. Do not swap them.
- For PageRank, mention that each link is a vote, links from high-rank pages count more, and the algorithm iterates until it **converges** (name the damping factor).
- State clearly that **client-side validation is not secure** — it can be bypassed, so security-critical checks must also run server-side.
- Compare client vs server on the three axes examiners reward: speed, server load, and security.